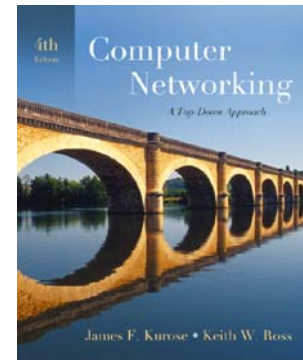


# Week 08

## Wireshark Lab: DNS

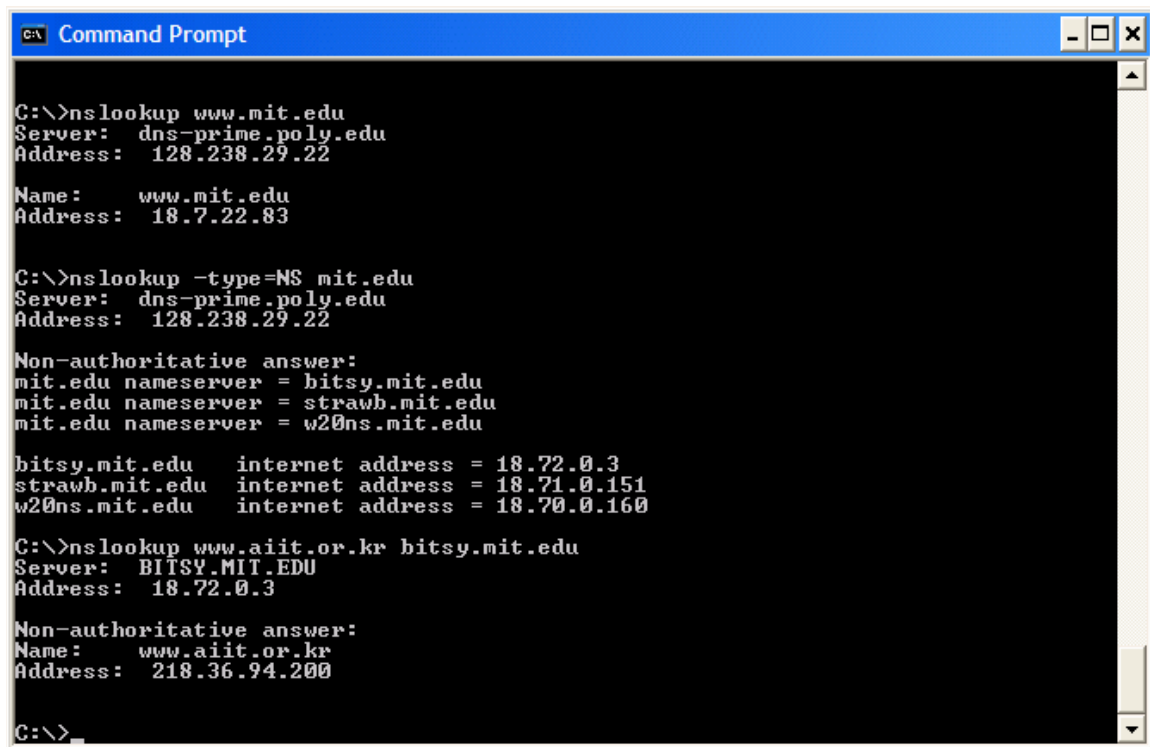
Please note that this exercise is designed for Windows machines. Change the commands accordingly if you are using Linux.



### 1. nslookup

In this lab, we'll make extensive use of the *nslookup* tool, which is available in most Linux/Unix and Microsoft platforms today. To run *nslookup* in Linux/Unix, you just type the *nslookup* command on the command line. To run it in Windows, open the Command Prompt and run *nslookup* on the command line.

In its most basic operation, *nslookup* tool allows the host running the tool to query any specified DNS server for a DNS record. The queried DNS server can be a root DNS server, a top-level-domain DNS server, an authoritative DNS server, or an intermediate DNS server (see the textbook for definitions of these terms). To accomplish this task, *nslookup* sends a DNS query to the specified DNS server, receives a DNS reply from that same DNS server, and displays the result.



```
C:\>nslookup www.mit.edu
Server:  dns-prime.poly.edu
Address:  128.238.29.22

Name:    www.mit.edu
Address:  18.7.22.83

C:\>nslookup -type=NS mit.edu
Server:  dns-prime.poly.edu
Address:  128.238.29.22

Non-authoritative answer:
mit.edu nameserver = bitsy.mit.edu
mit.edu nameserver = strawb.mit.edu
mit.edu nameserver = w20ns.mit.edu

bitsy.mit.edu    internet address = 18.72.0.3
strawb.mit.edu   internet address = 18.71.0.151
w20ns.mit.edu    internet address = 18.70.0.160

C:\>nslookup www.aiit.or.kr bitsy.mit.edu
Server:  BITSY.MIT.EDU
Address:  18.72.0.3

Non-authoritative answer:
Name:    www.aiit.or.kr
Address:  218.36.94.200

C:\>
```

The above screenshot shows the results of three independent *nslookup* commands (displayed in the Windows Command Prompt). In this example, the client host is located on the campus of Polytechnic University in Brooklyn, where the default local DNS server is *dns-prime.poly.edu*. When running *nslookup*, if no DNS server is specified, then *nslookup* sends the query to the default DNS server, which in this case is *dns-prime.poly.edu*. Consider the first command:

`nslookup www.mit.edu` // Linux: you can use `dig`

In words, this command is saying “Please send me the IP address for the host *www.mit.edu*.” As shown in the screenshot, the response from this command provides two pieces of information: (1) the name and IP address of the DNS server that provides the answer; and (2) the answer itself, which is the host name and IP address of *www.mit.edu*. Although the response came from the local DNS server at Polytechnic University, it is quite possible that this local DNS server iteratively contacted several other DNS servers to get the answer.

Now consider the second command:

`nslookup -type=NS mit.edu` // try `dig` if `nslookup` fails.

You can also use: `nslookup -type=ns mit.edu`

In this example, we have provided the option “-type=NS” and the domain “*mit.edu*”. This causes *nslookup* to send a query for a type-NS record to the default local DNS server. In words, the query is saying, “Please send me the host names of the authoritative DNS for *mit.edu*.” (When the `-type` option is not used, *nslookup* uses the default, which is to query

for type A records) The answer, displayed in the above screenshot, first indicates the DNS server that is providing the answer (which is the default local DNS server) along with three MIT name servers. Each of these servers is indeed an authoritative DNS server for the hosts on the MIT campus. However, *nslookup* also indicates that the answer is “non-authoritative,” meaning that this answer came from the cache of some server rather than from an authoritative MIT DNS server. Finally, the answer also includes the IP addresses of the authoritative DNS servers at MIT. (Even though the type-NS query generated by *nslookup* did not explicitly ask for the IP addresses, the local DNS server returned these “for free” and *nslookup* displays the result.)

Now finally consider the third command:

```
nslookup www.manchester.ac.uk 8.8.8.8
```

Now that we have gone through a few illustrative examples, you are perhaps wondering about the general syntax of *nslookup* commands. The syntax is:

```
nslookup -option1 -option2 host-to-find dns-server
```

In general, *nslookup* can be run with zero, one, two or more options. And as we have seen in the above examples, the dns-server is optional as well; if it is not supplied, the query is sent to the default local DNS server.

Now that we have provided an overview of *nslookup*, it is time for you to test drive it yourself. Do the following (and write down the results):

1. Run *nslookup* to obtain the IP address of a Web server in Asia.

**Ans.**

2. Run *nslookup* to determine the authoritative DNS servers for a university in Europe.

**Ans.**

3. Run *nslookup* so that one of the DNS servers obtained in Question 2 is queried for the mail servers for Gmail.

**Ans.**

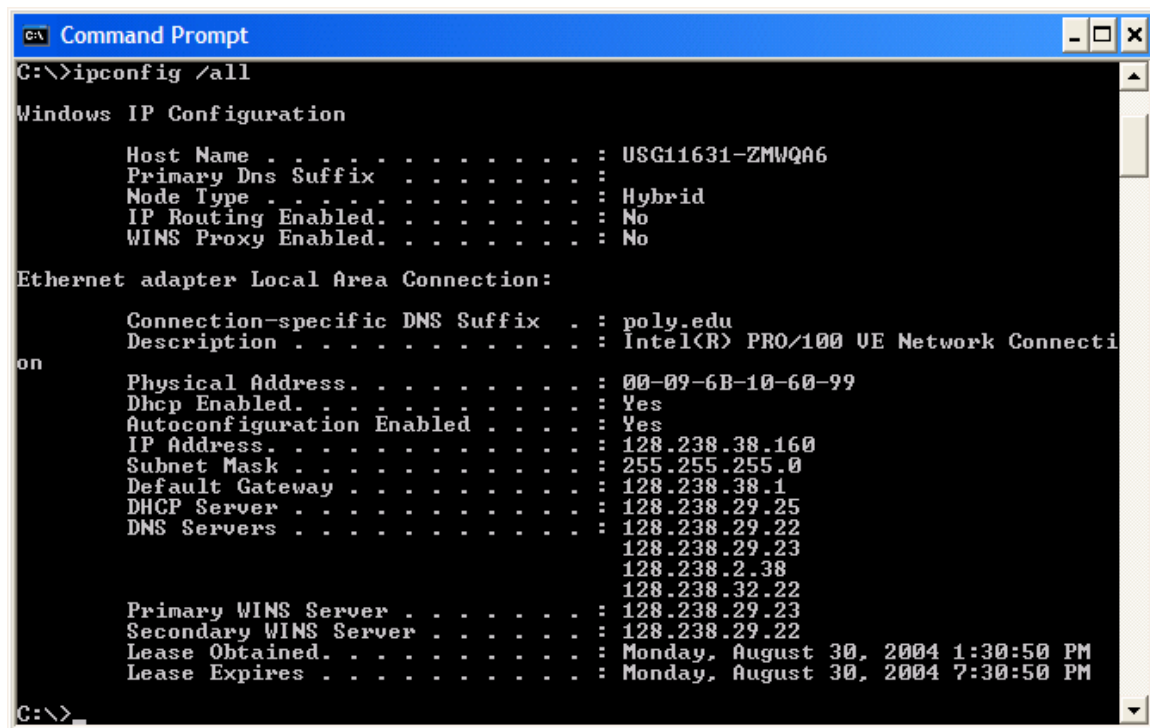
## 2. ipconfig

ipconfig (for Windows) and ifconfig (for Linux/Unix) are among the most useful little utilities in your host, especially for debugging network issues. Here we'll only describe ipconfig, although the Linux/Unix ifconfig is very similar. ipconfig can be used to show your current TCP/IP information, including your address, DNS server addresses, adapter

type and so on. For example, if you want to see all this information about your host, simply enter:

`ipconfig \all` // or `ifconfig` followed by

into the Command Prompt, as shown in the following screenshot.



```
C:\>ipconfig /all

Windows IP Configuration

    Host Name . . . . . : USG11631-ZMWQA6
    Primary Dns Suffix . . . . . :
    Node Type . . . . . : Hybrid
    IP Routing Enabled. . . . . : No
    WINS Proxy Enabled. . . . . : No

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix . : poly.edu
    Description . . . . . : Intel(R) PRO/100 VE Network Connection
    Physical Address. . . . . : 00-09-6B-10-60-99
    Dhcp Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    IP Address. . . . . : 128.238.38.160
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 128.238.38.1
    DHCP Server . . . . . : 128.238.29.25
    DNS Servers . . . . . : 128.238.29.22
                           128.238.29.23
                           128.238.2.38
                           128.238.32.22
    Primary WINS Server . . . . . : 128.238.29.23
    Secondary WINS Server . . . . . : 128.238.29.22
    Lease Obtained. . . . . : Monday, August 30, 2004 1:30:50 PM
    Lease Expires . . . . . : Monday, August 30, 2004 7:30:50 PM

C:\>
```

*ipconfig* is also very useful for managing the DNS information stored in your host. We learned that a host can cache DNS records it recently obtained. To see these cached records, after the prompt `C:\>` provide the following command:

`ipconfig /displaydns` // NOTE: This part cannot be done in Linux

Each entry shows the remaining Time to Live (TTL) in seconds. To clear the cache, enter

`ipconfig /flushdns` // Linux: `sudo service network-manager restart`

Flushing the DNS cache clears all entries and reloads the entries from the hosts file.

### 3. Tracing DNS with Wireshark

Now that we are familiar with *nslookup* and *ipconfig*, we're ready to get down to some serious business. Let's first capture the DNS packets that are generated by ordinary Web-surfing activity.

- Use *ipconfig* to empty the DNS cache in your host. // Linux: `sudo service network-manager restart`
- Open your browser and empty your browser cache. (With Internet Explorer, go to Tools menu and select Internet Options; then in the General tab select Delete Files.)
- Open Wireshark and enter "ip.addr == your\_IP\_address" into the filter, where you obtain your\_IP\_address (the IP address for the computer on which you are running Wireshark) with *ipconfig*. This filter removes all packets that neither originate nor are destined to your host.
- Start packet capture in Wireshark.
- With your browser, visit the Web page: **www.mahidol.ac.th**
- Stop packet capture.

If you are unable to run Wireshark on a live network connection, you can download a packet trace file that was captured while following the steps above on one of the author's computers<sup>1</sup>. Answer the following questions:

4. Locate the DNS query and response messages. Are they sent over UDP or TCP?

**Ans.**

5. What is the destination port for the DNS query message? What is the source port of DNS response message?

**Ans.**

6. To what IP address is the DNS query message sent? Use *ipconfig* to determine the IP address of your local DNS server. Are these two IP addresses the same?

**Ans.**

7. Examine the DNS query message. What "Type" of DNS query is it? Does the query message contain any "answers"?

**Ans.**

---

# Lab Exercise – TCP 3-Way Handshake

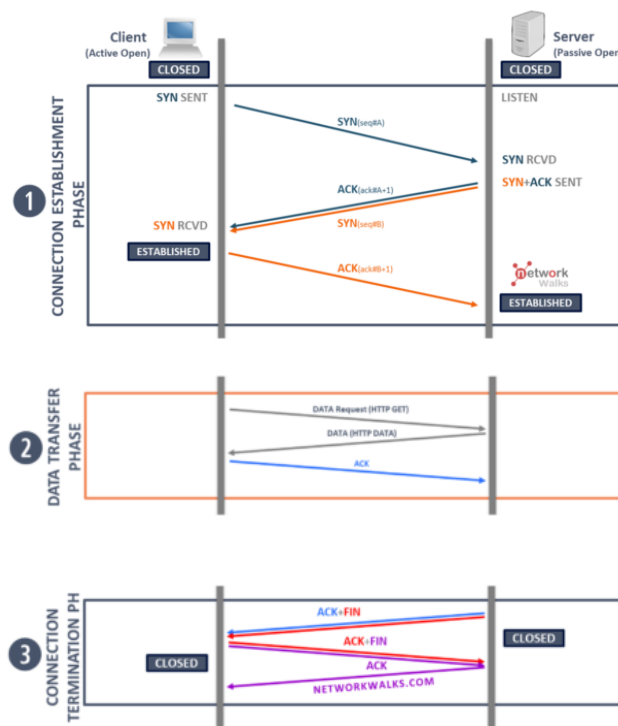
## Objective

The TCP 3-Way Handshake is a foundational process used in TCP/IP networks to establish a reliable, connection-oriented link between a client and a server before any actual data is exchanged.

**Step 1 (SYN - Synchronize):** The client initiates the connection by sending a TCP packet with the SYN (Synchronize) flag set and a random Initial Sequence Number, e.g.,  $\backslash(x\backslash)$ . This tells the server the client wants to communicate and establishes the starting point for the client's data stream.

**Step 2 (SYN-ACK - Synchronize/Acknowledge):** The server responds with both the SYN and ACK flags set. It acknowledges the client's packet by sending an Acknowledgement Number of  $\backslash(x + 1\backslash)$ , and provides its own random Initial Sequence Number, e.g.,  $\backslash(y\backslash)$ .

**Step 3 (ACK - Acknowledge):** The client receives the server's SYN-ACK packet and finalizes the connection by sending an ACK packet back to the server. The acknowledgement number is set to  $\backslash(y + 1\backslash)$ .



## Inspect the TCP 3-Way Handshake

### 1. access to any Website

This filter will show TCP 3-way handshake. Try to find all TCP Message Types

The screenshot shows a Wireshark capture of a TCP 3-way handshake. The packet list at the top shows the following packets:

| No. | Time     | Source   | Destination | Protocol | Length | Info                                                                       |
|-----|----------|----------|-------------|----------|--------|----------------------------------------------------------------------------|
| 1   | 0.000000 | 10.0.0.9 | 10.0.0.3    | TCP      | 66     | 53076→80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1            |
| 2   | 0.001081 | 10.0.0.3 | 10.0.0.9    | TCP      | 66     | 80→53076 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1260 WS=256 SACK_PERM=1 |
| 3   | 0.001226 | 10.0.0.9 | 10.0.0.3    | TCP      | 54     | 53076→80 [ACK] Seq=1 Ack=1 Win=66560 Len=0                                 |
| 4   | 0.001568 | 10.0.0.9 | 10.0.0.3    | HTTP     | 430    | GET / HTTP/1.1                                                             |
| 5   | 0.005353 | 10.0.0.3 | 10.0.0.9    | TCP      | 324    | [TCP segment of a reassembled PDU]                                         |
| 6   | 0.057855 | 10.0.0.9 | 10.0.0.3    | TCP      | 54     | 53076→80 [ACK] Seq=377 Ack=271 Win=66304 Len=0                             |
| 7   | 0.058967 | 10.0.0.3 | 10.0.0.9    | HTTP     | 218    | HTTP/1.1 200 OK (text/html)                                                |
| 8   | 0.112365 | 10.0.0.9 | 10.0.0.3    | TCP      | 54     | 53076→80 [ACK] Seq=377 Ack=435 Win=66304 Len=0                             |
| 9   | 4.190623 | 10.0.0.3 | 10.0.0.9    | TCP      | 60     | 80→53076 [FIN, ACK] Seq=435 Ack=377 Win=66560 Len=0                        |
| 10  | 4.190704 | 10.0.0.9 | 10.0.0.3    | TCP      | 54     | 53076→80 [ACK] Seq=377 Ack=436 Win=66304 Len=0                             |
| 11  | 4.908232 | 10.0.0.9 | 10.0.0.3    | TCP      | 54     | 53076→80 [FIN, ACK] Seq=377 Ack=436 Win=66304 Len=0                        |
| 12  | 4.911047 | 10.0.0.3 | 10.0.0.9    | TCP      | 60     | 80→53076 [ACK] Seq=436 Ack=378 Win=66560 Len=0                             |

The packet details pane for the first packet (SYN) shows the following information:

- Source Port: 53076
- Destination Port: 80
- [Stream index: 0]
- [TCP Segment Len: 0]
- Sequence number: 0 (relative sequence number)
- Acknowledgment number: 0
- Header Length: 32 bytes
- Flags: 0x002 (SYN)
- Window size value: 8192
- [Calculated window size: 8192]
- Checksum: 0x4f08 [unverified]
- [Checksum Status: Unverified]
- Urgent pointer: 0
- Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted

a) What is an Initial Sequence Number (also show a captured screenshot)?

Ans.

b) What is a Sequence and Ack Number (also show a captured screenshot)?

Ans.

c) What is an Ack Number (also show a captured screenshot)?

Ans.

